Elasticsearch: Aggregated DataLake

Bhavin Mandiwala

What is Elasticsearch?

A distributed, RESTful search and analytics engine built on Apache Lucene. You send JSON documents over HTTP; it indexes every field so you can search and aggregate in near real time.

1 Search

Full-text, exact match, ranges, fuzzy, geo; results ranked by relevance

2 Analytics

Aggregations: counts, averages, histograms, top-N, cardinality

3 Distributed

Data split into shards, copied as replicas, spread across nodes

4 Near real time

A new document becomes searchable about one second after indexing

Building blocks and terminology

Elasticsearch	Relational analogy	Meaning
Cluster	Database server farm	Group of nodes sharing one name and one data set
Node	Server process	One running Elasticsearch instance (JVM)
Index	Table	Collection of documents with a shared mapping and settings
Document	Row	A JSON object stored in an index, identified by _id
Field	Column	A key in the document; each has a type
Mapping	Schema	Defines field types and how each is indexed
Shard	Partition	A slice of an index; each shard is a Lucene index
Replica	Standby copy	Copy of a primary shard for failover and read capacity
Segment	Immutable file set	Unit of storage inside a shard, created at refresh

Inverted index

Documents (forward view)

Doc 1: fast delivery please

Doc 2: please call before delivery

Doc 3: fast refund

A book index works the same way: instead of scanning every page for a word, you look the word up and get the page numbers.

Inverted index (term to documents)

term	postings
before	2
call	2
delivery	1, 2
fast	1, 3
please	1, 2
refund	3

Query "fast AND delivery": look up both terms, intersect postings 1,3 and 1,2, result is Doc 1. No full scan.

Documents and mappings

Index a document

```
PUT /orders/_doc/1
{
  "customer": "Asha Rao",
  "status": "shipped",
  "total": 249.99,
  "placed_at": "2026-10-01",
  "note": "fast delivery please"
}
```

Explicit mapping

```
PUT /orders
{ "mappings": { "properties": {
  "customer":{"type":"keyword"},
  "status":  {"type":"keyword"},
  "total":   {"type":"double"},
  "placed_at":{"type":"date"},
  "note":    {"type":"text"}
}}}
```

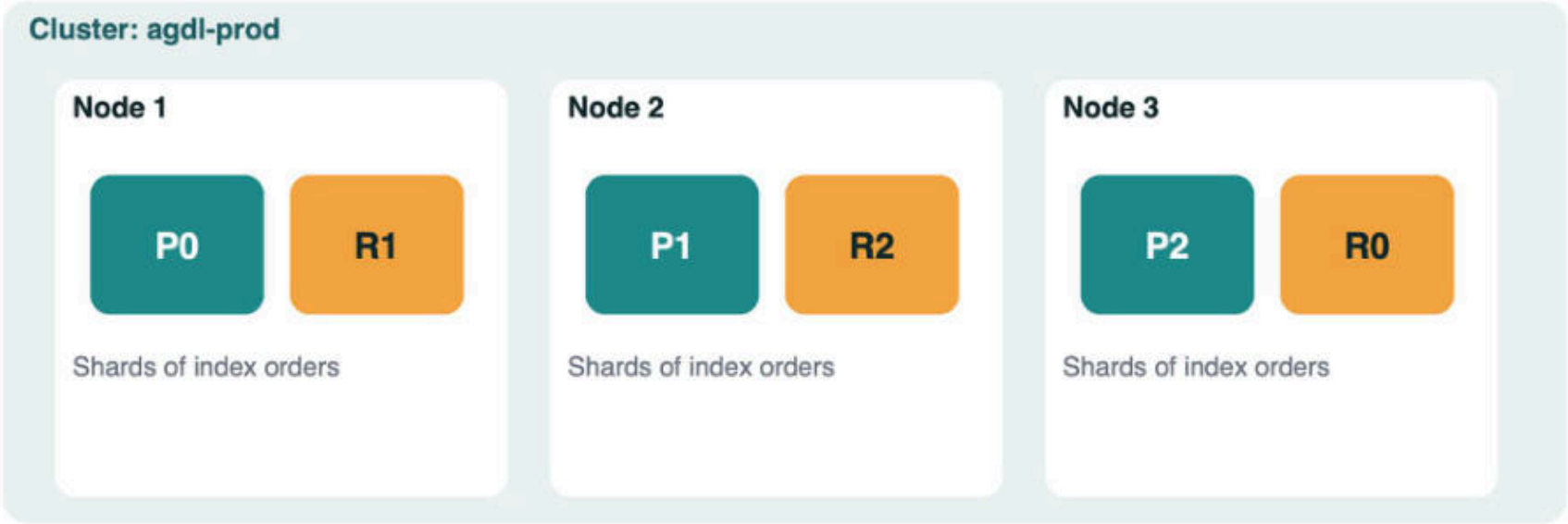
keyword: exact value, used for filters, sorting and aggregations. **text**: analyzed into terms for full-text search. Prefer explicit mappings; dynamic mapping guesses types and can bloat the index.

Text analysis: from text to terms



The same analyzer runs at index time and (for match queries) at search time, so "Delivery" finds "delivery". keyword fields skip analysis and store the exact value.

Cluster, node, index, shard



Index **orders** has 3 primary shards (P0 to P2), each with 1 replica (R). A replica never sits on the same node as its primary, so one node can fail without data loss.

Node roles

Master-eligible

Holds cluster state: index metadata, shard allocation. One elected master. Keep it light: no heavy queries.

Data (hot / warm / cold)

Stores shards, runs indexing, search and aggregations. Most CPU, RAM and disk go here.

Ingest

Runs ingest pipelines (grok, rename, enrich) before indexing.

Coordinating only

Routes requests, scatters to shards, merges results. All nodes coordinate; dedicated ones offload heavy aggs.

Machine learning / transform

Optional roles for anomaly detection and transforms.

Voting-only

Takes part in master elections without ever becoming master.

Shard = Lucene index = segments



A segment is an immutable, self-contained mini index: its own inverted index, doc values, stored fields. Each refresh writes a new small segment; background merges combine small segments into bigger ones.

Search

Visits all segments of the shard and merges hits

Delete / update

Marks the old doc deleted; new version goes in a new segment

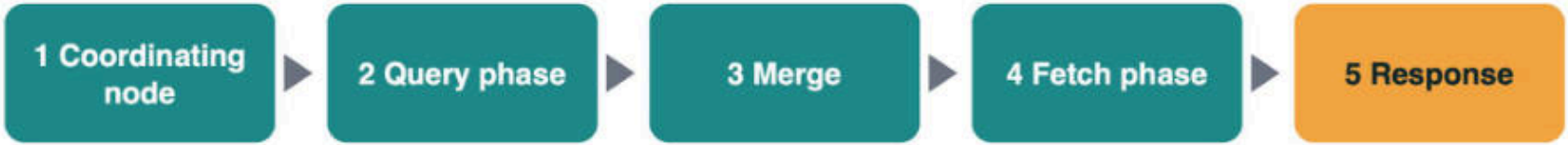
Merge

Rewrites segments, drops deleted docs, keeps segment count low

Real-time ingestion: the write path



Query flow: scatter and gather



Receives the search, picks one copy (primary or replica) of every shard

Each shard runs the query over its segments, returns top from+size doc IDs and scores

Coordinator merges shard results into a global top list

Asks only the owning shards for the _source of the winning docs

Hits, aggregations and timings returned

What is a data lake?

A central store that keeps large volumes of raw data in its original form, to be shaped when it is used.



Any data

structured, semi-structured, unstructured

Low-cost scale

cheap object storage, add capacity as needed

Schema on read

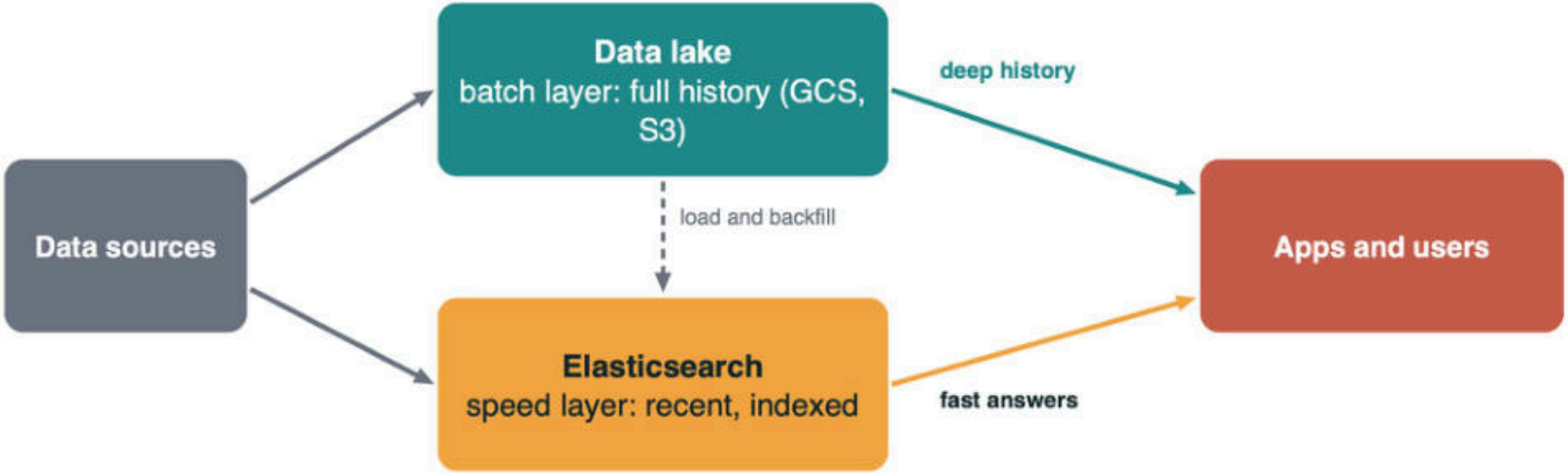
structure applied when data is queried

Separate compute

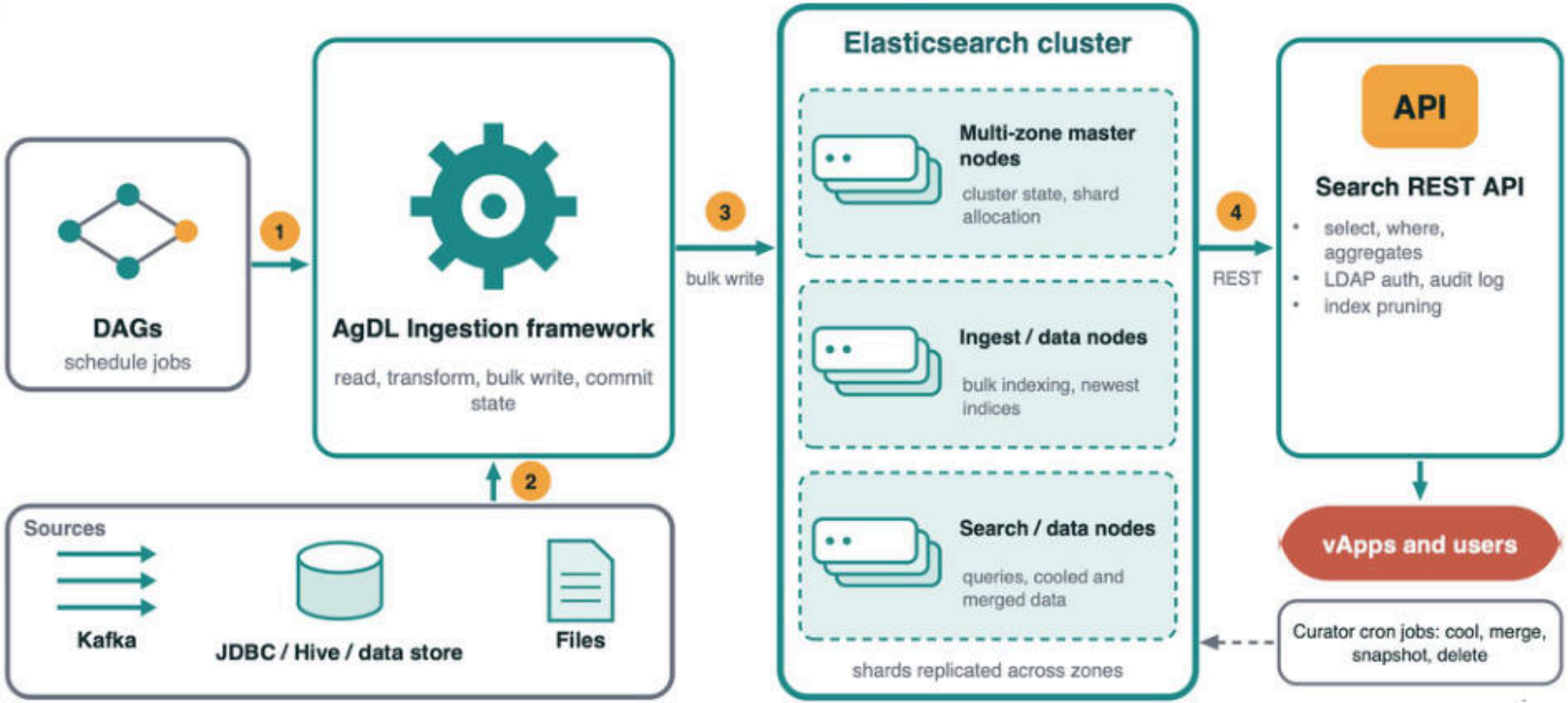
many engines read one copy

Elasticsearch: the speed layer next to the lake

Speed layer: the fast path that answers queries on recent data in seconds while the lake keeps everything.



AgDL at a glance

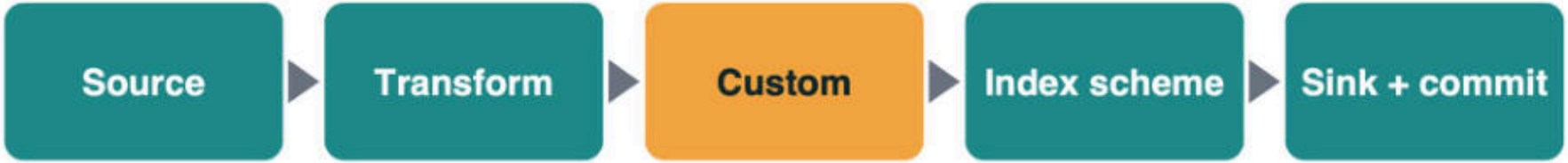


How Elasticsearch fits the AgDL use case

What AgDL needs	Elasticsearch capability	How AgDL uses it
Event data from many sources, arriving continuously	**Bulk writes in parallel; time-based indices**	Ingestion framework builds daily or monthly indices
Look up an indicator (hash, URL, IP) in seconds	**Inverted index term lookup plus filters**	select and where on searchable attributes
Find fragments of paths, URLs, command lines	**Tokenised sub-fields for text matching**	kw_search on token fields
Instant top values, distinct counts, trends	**Aggregations on columnar doc values**	TOPN, COD, HITSOVERTIME, MIN / MAX
Recent data fast, old data cheap, bounded retention	**Hot and search tiers, snapshots, index deletion**	Curator cools, merges, snapshots, deletes by retention

AgDL queries are mostly time-bounded lookups, filters and summaries over event data: exactly what indexes, doc values and time-based partitions are built for.

The ingest pipeline



Ingestion framework: what a configuration holds

Source

- type: Kafka batch, table, SQL query, JDBC or files
- connection details
- where progress is tracked

Transformations

- one rule per output field
- source name, target name, converters
- output columns = the listed targets

Sink

- index prefix
- scheme: single, daily, monthly
- date column that names the index
- cluster address, bulk size

Engine

- application name
- memory and cores
- run mode: local or cluster

Built-in converters (named in the transformation rules)

- cast to string
- strip time
- strip milliseconds
- host from URL
- bad value to null
- decode
- normalise text
- custom step

Add a new source with configuration only

web_events.properties (illustrative; key names simplified)

```
source.type              = KAFKA_BATCH
source.kafka.brokers     = <broker:port>
source.kafka.topic       = web_events
source.state.path        = <state location>
source.transformations = [
  {"sourceName":"url","destName":"url",
   "converters":["CAST_STRING"]},
  {"sourceName":"url","destName":"url_domain",
   "converters":["GET_HOST_NAME"]},
  {"sourceName":"event_ts","destName":"event_date",
   "converters":["STRIP_TIME"]} ]
sink.type                = ES
sink.es.index_prefix     = web_events_
sink.es.index_scheme     = DAILY
sink.es.index_date_col   = event_date
es.nodes = <cluster host>   es.port = 9200
```

1. **Write the config**
   source, field rules, sink, as shown
2. **Add an index template**
   mappings, shards, replicas for the new indices
3. **Schedule it**
   one scheduler entry that points at the config
4. **Register for search**
   one entry in the query metadata: prefix, scheme, attributes

Custom code is only needed when a field needs logic beyond the built-in converters.

Ingesting at lake scale

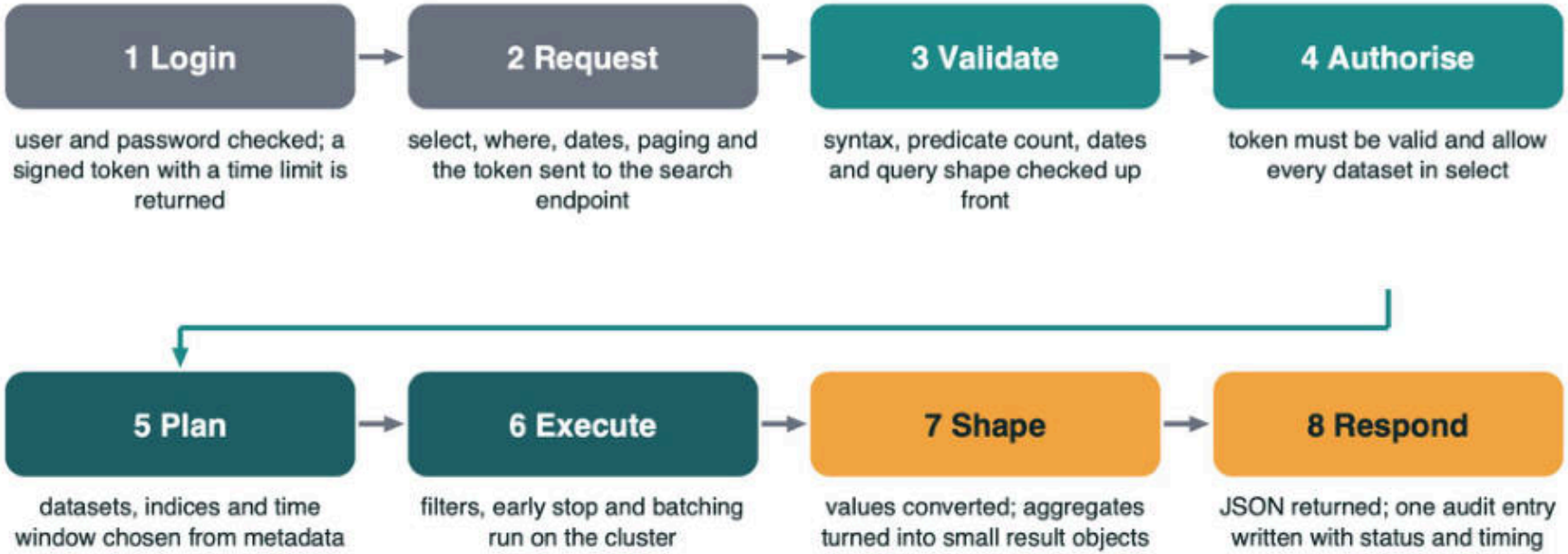
Keeping ingestion fast

- Time-based indices (daily or monthly): new data hits only the newest indices
- Index templates apply mappings and settings automatically
- Bulk batches of a few MB with parallel workers
- Relax refresh_interval during large backfills, then restore it
- Ingest pipelines for light enrichment close to the data

AgDL Ingestion framework

- Spark jobs read Hive, Kafka or files and transform fields from configuration
- elasticsearch-spark writes partitions to Elasticsearch in parallel
- Index name derived per row: single, daily or monthly
- State is committed only after a successful write, so a failed run is retried
- Lag metrics reported to monitoring

API workflow: from login to response



Errors are typed: 400 invalid query, 401 not authorised, 504 timeout, 500 other.

Where each optimisation sits in the request path



Search newest first, stop early

A dataset with daily indices and 6 months of retention: 184 indices; batch size 30 days; limit 10



enough rows? stop

If batch 1 returns 10 rows, the other 154 indices are never searched: about 16% of the window touched.

Best case

recent activity fills the limit in batch 1

Typical

finishes in the first batches

Worst case

no matches: every batch runs in turn

What goes in and what comes out

Request field	Purpose
`select`	what to return: DATASET.column or AGG(DATASET.column)
`where`	SQL-style filter on searchable columns
`startDate / endDate`	time window; chooses the indices
`start / limit`	paging inside each dataset
`kw_search`	token match on text columns
`uid`	fetch specific records by id
`authToken`	session token from login

Plain query response

```
{ "requestId": "r-1",
  "results": {
    "DATASET": {
      "matches": 482,
      "values": [ {...}, {...} ] } } }
```

Aggregate query response

```
{ "requestId": "r-2",
  "results": {
    "COD(DATASET.col)": {"distict_count": 1840},
    "TOPN(DATASET.col:0:3)": {"values": [
      {"value":"a","count":420}, ... ]} } }
```

Plain and aggregate queries are separate requests: results are keyed by dataset or by the aggregate you asked for.

Important design points

Metadata-driven

One metadata file describes every dataset: index prefix, scheme, retention, columns, limits. New datasets are mostly configuration.

SQL-style language

select, where and dates map to familiar SQL; a grammar parses where into filters; limited on purpose.

Layered modules

API contracts, query engine and REST service are separate; the service only sees interfaces.

Stateless sessions

A signed, expiring token carries identity and allowed datasets; no server-side session store.

Strategy by dataset

Each dataset can use the default executor or a specialised one (batched, latest-only).

Auditable and typed

Every request writes one audit entry; failures map to clear status codes.

Performance optimisations

Prune indices

Time window, retention and start date turn into a short index list before any search.

Prune datasets

Datasets that cannot satisfy the where columns are skipped, with a fast 400 if none can.

Filter context

Conditions run as cacheable yes/no filters; no relevance scoring.

Stop early

terminate_after and right-sized limits end each shard's work as soon as enough rows are found.

Read less

Only selected columns are read, from columnar doc values; aggregations return no documents.

Fewer round trips

Searches are batched into one multi-search request; connections are pooled.

Questions?

Thank you for your time.

Bhavin Mandiwala

Let's connect

